

经历十年，对数学教学的一些思考

Igor Rivin

几乎恰在 10 年前，我在 Temple 大学网站上首次发表了一个“宣言”，本文多少是它的扩充和修正版。从那时起，我把某些在下文中表述的想法付诸实践——对我的 10 年经历的简要描述见下面“我在过去 10 年间的经验”一节——我也收到了相当多的反馈意见。某些（大多数）意见完全是正面的；另一些则包含有带批判性的想法。我在“一些反馈”那节描述了其中的某些反馈。

1. 问题是什么？

公立大学的数学教授身负着许多责任，其中包括研究工作，行政事务和教学，继而又包括“专业型”教学（对象是主修数学的学生和研究生）和“服务型”教学：教一、二年级学生的数学。我的这些想法主要集中于服务型教学，对我而言，它把我工作中一些最令人兴奋和最令人郁闷的事情结合在了一起。在“我们能做些什么？”一节中，加进了关于教主修数学的学生的一些想法。

a. 产品

为什么郁闷？请看下列事实：绝大多数服务型课程都是关于微分和积分演算，以及线性代数的。这两门课是相当深的，下述事实就是明证：数学在极有才华的人们经过数千年的实践，微积分的基本原理才在 17 世纪被建立起来（尽管有些原理已被 Archimedes（阿基米德）用特别的方式发现——在相当早的时代）。为了使这门学科建立在真正牢固的基础上，人们又做了大量研究，花费了 200 年时间。今天使用的线性代数成熟得更晚。流行的矩阵方法和线性变换技巧到 20 世纪初才发展成真正现代的形式。我们别无选择，只能承认这些学科是相当深刻的，需要相当专门的技能才能成功地使用它们。

b. 消费者

我们向谁教授这些学科呢？在公立大学（如 Temple 大学），我们学生的基本水平，多少处于美国公立中等教育系统毕业生的平均水平之上。这意味着带有分数的算术技巧已对他们形成相当大的压力。他们基本没有抽象推理的能力，而他们与证明的概念也形同陌路。

c. 结果

其结果是，对他们，我们几乎不可能教我们意欲教的东西：高等数学以一定程度的抽象性为前提，即使我们罪过地忘记它，并把微积分定义为“无需去理解的一堆计算技

译自：Notices of the AMS, Vol. 61 (2014), No. 6, p. 597-602, Some Thoughts on the Teaching of Mathematics—Ten Years Later, Igor Rivin. Copyright ©2014 the American Mathematical Society. Reprinted with permission. All rights reserved. 美国数学会与作者授予译文出版许可。

Igor Rivin 是 Temple 大学的数学教授和布朗大学的 ICERM 访问教授。他的邮箱地址是 igor.rivin@temple.edu。

巧的汇集”，由于学生技术上的弱点，在这方面基本上也教不出有价值的结果。他们不能计算。结果是，我们的微积分和线性代数课就是一堆平凡例子的汇集，学生必须靠死记硬背来记住它们。这样做的后果除了让学生对数学产生害怕和厌恶之外，没教会他们任何东西。更何况大多数学生在他们未来的生活中绝不会去使用微积分（这种现象不足为奇，因为正如上面所述，他们对微积分什么都不知道），甚至可以说，他们在上这些课之前也从未有过使用高等数学的任何打算。要求他们上这些课是由于这样的信念（不无道理）：对所有受过高等教育的人而言，数学应该是他们智力结构中的一部分。结果是，厌恶数学的观念进入了相当大的多数美国人的智力结构中。令人吃惊（也令人高兴）的是，尽管有上述的现象，我们仍看到有些学生居然在勉力理解该学科中的某些内容。然而，这种高兴被两种想法冲淡了：一种想法是它浪费了大量时间，二是如果教法得当，这些有才能的学生会收获些什么呢！¹⁾

2. 我们能做些什么？

那么，答案是什么呢？我们能舍弃数学中的分类要求（我能很容易地看到它的发生）；实际情况是逻辑和抽象推理能力应该是每个接受高等教育的他或她要得到的（也许是主要的）东西，也是我们作为数学教育工作者（我们就是这样的人，这肯定也是大多数本能的反应）应该徐徐教导给学生的东西。怎么做呢？首先是向后退一步——从“高等”数学，即无限大和无限小的数学——退到概念上简单的数学形式。

在（并不十分遥远的）过去，Euclid（欧几里得）几何就是准确按这样的理念教学的一门学科（直线，圆，距离等等基本概念相当直观，数学推理的基本组件也都出现在其中）。当然，我未必会提倡回到这门学科上来。首先，这门学科失去活力已有几百年了；其次，它离现代美国人的实践经验相距甚远。

作为替代，我的意见是我们在一开始就应该讲推理（逻辑），计数（它意指朴素集合论、组合学和图论）和概率论。由于计算机无处不在，这些学科在我们生活中有了明显的重要性。它们既比较容易学，也比我们目前教的东西让学生更直接地得到回报。此外，这里还有另一个信条，它们至少能用来清除学生头脑中某些糊涂观念。这个信条是：

计算机程序是证明。 (1)

例如，学生写出一节程序“Scheme 语言下的模算术软件包的源代码”，他必须完全理解中国剩余定理，他的程序就是中国剩余定理：只要给定一些满足定理假设的量，它肯定会产生出满足结论的量。

程序和证明之间的这种对应（尽管离完美还相差甚远），允许我们对数学有一种实地的感受（给计算机编程能快速得到反馈，正反馈或负反馈）；这就跟学生的经验紧密联系上了，显然是“有益的”。（当然，真正实用的数学程序处于更深层次，但...）

这种做法的一个问题是，可能要浪费很多时间向学生介绍某些（可能有专利的）编程语言或科学计算系统的特征与微妙之处。重要之处在于，让学生一开始就接触数学上简洁

1) 在咖啡时间的典型对话会这样开场：非数学家：你是做什么的？数学家：我研究数学。
非数学家（两位问答者之一）：我很怕数学（或）我一向喜欢数学！这是非常美国式的现象；我在欧洲从未见过这样的对话。——原注

的系统, 乐于老是想着数学的 抽象机器 (abstract machine). 很幸运, 这样的系统唾手可得, 它在麻省理工学院, 印第安纳大学和其它多所学校的计算机科学教育中已用了 30 多年. 这就是 Scheme 编程语言.¹⁾绝对值得称誉的一本书是 H. Abelson 和 G. J. Sussman 撰写的《计算机程序的结构和解释 (Structure and Interpretation of Computer Programs)》[1], 该书介绍了计算机编程的基本原理; 与之相伴的书是《数学的结构和解释 (Structure and Interpretation of Mathematics)》²⁾, 它可以算作是新式的介绍数学的核心部分. 应该指出, Scheme 编程语言在麻省理工学院不再作为学习计算的入门课程, 取而代之的是 Python 语言, 这么做的理由是后者在实际工作中更实用 (对几乎所有你可能需要完成的任务, 都有相应的程序库), 而且它已吸收了大部分 Scheme 语义. 然而, 把它当做首选语言不甚妥当, 因为它不够纯且较少严格定义的语义.

考虑到 Python 语言, Mathematica 语言和其它“一大堆”语言具有非常高级别的功能, 那么, 作为折中方案, 开始时先讲 Scheme 语言来打基础, 然后当 (更精确地说是“如果”) 需要引入数学中更高深的课题时, 再使用那些语言中的一种.

3. 可能的异议

对上述的方案, 你能预料到会有某些反对意见:

a. 微积分该怎么办呢?

没有人提议要从大学课程中完全取消微积分: 科学和工程方面的学生 (不要说数学方面的学生了) 确实需要熟悉它. 然而, 我们也许应稍晚一些, 等学生在数学思维方面更成熟些, 等他们真的更能理解这一主题内容的时候再教他们. 实际情况是, 不少学生将被要求学习微积分; 另一方面, 学习数学的人数可能也确实在增加.

b. 教师又该怎么办?

据我的观察, 许多接受过“传统”方法教育的助教, 对“有限”数学有点荒疏. 对许多教授而言, (有话直说!) 情况也一样. 许多教师和研究生对计算机编程根本不熟悉. 第 1 个问题很容易搞定, 第 2 个就稍麻烦些, 但我的看法是凡对新手可教的东西, 要教会教师 (这对他们有用) 应该更容易.

4. 我在过去 10 年间的经验

这个文件的第 1 版发表以来, 我已教过许多相关的课程 (有的还重复教过). 有门课叫“初级问题求解 (Junior Problem Solving)”, 这是为主修计算机科学的学生开的数学入门课程. 另一门是“数学模式 (Mathematical Patterns)”, 这类数学是为“诗人课程 (Poets course)”准备的. 第 3 门是“高级问题求解 (Senior Problem Solving)”, (原则上) 它是 Temple 大学数学系学生的最后一门数学课. 第 4 门是“数学计算 (Mathematical Computing)”——是为数学方面的研究生开的课.

a. 诗人课程

在前两门课程 (初级问题求解; 数学模式) 中, 我教逻辑基础, 内容与 Harry Gensler

1) 更多细节见 <http://scheme.org>.——原注

2) 撰写这样的书是我非常喜爱的一个计划, 但这必须与在教学中使用这些想法同时进行, Abelson 和 Sussman 的书作为讲义发行了好几年.——原注

那本优秀的书 [2] 所介绍的一样, Gensler 是位伦理哲学家而非数学家. 特别要说明, 该书虽然最终详细描述了符号逻辑, 但书中有大量的问题涉及确定英语句子或 (更一般的) 哲学论证有效性的判定. 该课程在这两方面都是成功的: 在课程开始时 没有 学生能进行逻辑论证, 而在课程结束时 很多 学生都行了 (事实是, 他们在开始时并不认可我的信念——学生们尚未准备好去学习类似于微积分的任何课程). 这里再附加几条评注:

- 在初级问题求解中的某一处, 我决定试试数学味更浓的问题, 提出了 $\sqrt{2}$ 是无理数的证明. 尽管讲得很慢, 但很清楚涉及其中的算术对学生而言还是太多了.
- 在这门课结束时, 许多学生都感谢我 (有一次, 我走在费城中心城区的一条大道上, 我过去的一名学生要横穿马路, 正等待一些快速行驶的汽车驶过, 她告诉我她非常欣赏我课上的内容). 这类事情在我以前任何一次教微积分课结束后从未发生过.

b. 初露头角的数学家

在研究生课程方面, 我一开始就讲 Abelson-Sussman 的基础编程, 并强调跟数学问题的结合. 这方面的经历肯定是双峰式的. 好的学生 (我是指数学潜力大的学生) 具有很强的编程能力; 不太好的学生基本上没有这种能力. 当然, 这是相当不幸的, 因为 Temple 大学的博士候选者会进入工业界, 编程能力对他们未来的生计事关重大; 而最好的学生大都可能会追求在纯数学领域工作, 从某种意义上讲, 他们需要的编程能力最少 (虽然如 Gauss (高斯) 清楚知道的, 数学是一门实验科学, 而计算就是实验).

高级问题求解课程, 在某种程度上是最令人失望的——有些学生表现得特别优秀, 大多数学生则并不比计算机科学方面的学生在初级问题求解课上学到的技巧好多少 (更令人沮丧的是, 这些学生通常是主修数学教育的), 他们会让证明给搞糊涂了, 特别是那些需要进行计算的证明. 我十分肯定, 假如要求他们上一门如我给他们的富于想象力的同道开的那类课程, 加上严格的代数课程 (如我们这些在苏联成长起来的人在 12 岁时学的那类课), 他们也许能达到更好的数学程度. 然而, 当我教他们的时候为时已晚.

5. Scheme 语言下的模算术程序包的源代码

下述源代码 (逐字逐句取自我课上的一名学生做的家庭作业) 定义了所有 Scheme 语言下的模数学 (大部分都已有了, 而这样的程序其长度大大缩短了, 并在实践中更有效). 在结尾部分, 给出了实现扩展的 Euclid 算法的做法, 紧接着的是实现中国剩余定理算法的做法.

```
(define (intMod k m)
  (if (= m 0)
      (lambda (s) 0)
      (let* ((j (remainder k m))
              (l (if (< j 0)
                      (+ j m)
                      j)))
          (lambda (s)
            (if s
                l
                m))))))
```

```

(define (sameMod? x y)
  (= (x #f) (y #f)))

(define (eqM? x y)
  (and (= (x #f) (y #f))
        (= (x #t) (y #t))))

(define (displayM x)
  (display (x #t))
  (display "mod.")
  (display (x #f))
  (display "\n"))

(define (+M x y)
  (if (sameMod? x y)
      (intMod (+ (x #t) (y #t)) (x #f))
      (intMod 0 0)))

(define (-M x y)
  (if (sameMod? x y)
      (intMod (- (x #t) (y #t)) (x #f))
      (intMod 0 0)))

(define (*M x y)
  (if (sameMod? x y)
      (intMod (* (x #t) (y #t)) (x #f))
      (intMod 0 0)))

(define (Modulus x)
  (x #f))

(define (Coset x)
  (x #t))

(define (gcde a b)
  (letrec ((aux (lambda (x1 x2 y1 y2 r1 r2)
                  (let ((r3 (remainder r1 r2)) (q (quotient r1 r2)))
                    (if (= r3 0)
                        (list r2 x2 y2)
                        (aux x2
                           (- x1 (* q x2))
                           y2
                           (- y1 (* q y2))
                           r2
                           r3))))))
    (aux 1 0 0 1 a b)))

(define (/M x y)
  (let ((z (apply gcde (list (y #t) (y #f)))))
    (if (and (sameMod? x y) (= 0 (remainder (x #t) (car z))))
        (intMod (+ (cadr z) (/ (x #t) (car z))) (x #f))
        (intMod 0 0))))

(define (crt l)
  (if (null? (cdr l))
      (car l)
      (let* ((w (map Coset (list (cadr l) (car l))))
              (x (map Modulus (list (car l) (cadr l))))
              (y (apply gcde x))
              (z (car y)))
        (if (apply = (map remainder w (list z z)))
            (crt (cons
                  (intMod (/ (apply + (map + w x (cdr y))) z)
                          (/ (apply + x) z))
                  (cddr l)))
            (intMod 0 0)))))

```

6. 一些反馈

几乎所有的反馈都充满了积极的赞同声。下面举一些例子以及我自己的评论 ([IR]):

IR: 有些人建议, 概率论适合作为离散数学的导论. (普度大学的) Ward 和 Gundlach 已为此目的写了一本书.

• 我读了你的抨击性言论, 发现它十分令人信服——事实上, 我同意这些说法, 并发现跟我自己作为一名大学生时 (主修电子工程) 的经验相一致. 我在大学二年级上了离散数学课才开始理解数学. 我现在在 Caltech (加州理工学院) 研究应用数学, 可以这么说, 离散数学是我职业道路上一次重大转变的开始, 这绝不是随便说的.

同时, 你将体会到大量的工程师的感受, 他们感到缺乏诸如三维空间的向量分析的知识, 会严重地妨碍学生学习 (比如) 电磁工程. 尽管我上过这门课, 表面上 “知道” 其素材, 但仍发现实情就是如此.

[IR]: 当然, 我确实同意应该学习多元分析, 但不是第一位的事情.

IR: 有人建议, “综合的方法” (在其中, 并行地利用数学和使用数学的科学) 不失为我正在设法解决的某些问题的一种解法. 我对此的看法是, 这虽不失为一个好的想法, 但它还不能并行地教人去这样思考.

• (来自 Temple 大学的一位化学家): 嗨, 我已读过你抨击数学教育的文章的第 1 部分, 我同意你在其中所说的一切. 我觉得用同样的方法教我喜欢的有机化学课, 与其说是在教课, 不如说是在识别能够掌握和使用有机化学的 5% 的学生.

• (来自一位搞应用数学的朋友): 我问你: 始于 8 岁的最初阶段的教育体制价值几何?

[IR]: 我的回答是: 很少.

好. 那么我们能做得更好些吗? 回答是肯定的. 看看谷歌 Montessori (蒙台梭利) 学校. 我的两个孩子从 3—12 岁是在 Montessori 学校度过的. 当我试图向他们解释任何逻辑论证时, 他们看我的神态就好像我完全是个笨蛋, 因为在 Montessori 体制中, 进行逻辑推理对 12 岁的孩子已是很自然的事了.

参考文献

- [1] Harold Abelson and Gerald Jay Sussman, Structure and Interpretation of Computer Programs, Taylor & Francis.
- [2] Harry J Gensler, Introduction to Logic, Routledge, 2010.

(冯绪宁 译 袁向东 校)